

Object Oriented and Programming

Lab 5
INHERITANCE (Java)

Objectives

- Inheritance
- Implementing inheritance in Java

Inheritance

- Inheritance can be defined as the process where **one object acquires the properties of another.**
- With the use of inheritance the information is made manageable in a **hierarchical order.**
- When we talk about inheritance the most commonly used keyword would be **extends and implements.**
- **The class which inherits the properties of other is known as subclass (derived class, child class) and the class whose properties are inherited is known as superclass (base class, parent class).**

IS-A Relation

```
public class Animal{  
}  
  
public class Mammal extends Animal{  
}  
  
public class Reptile extends Animal{  
}  
  
public class Dog extends Mammal{  
}
```

- Now based on the above example, In Object Oriented terms following are true:
- Animal is the superclass of Mammal class.
- Animal is the superclass of Reptile class.
- Mammal and Reptile are sub classes of Animal class.
- Dog is the subclass of both Mammal and Animal classes.
- Now if we consider the IS-A relationship we can say:
- Mammal IS-A Animal
- Reptile IS-A Animal
- Dog IS-A Mammal
- Hence : Dog IS-A Animal as well

HAS-A Relationship

- These relationships are mainly based on the usage. This determines whether a certain class HAS-A certain thing. This relationship helps to reduce duplication of code as well as bugs.

```
public class Vehicle{}  
public class Speed{}  
public class Van extends Vehicle{  
    private Speed sp;  
}
```

This shows that class Van HAS-A Speed. By having a separate class for Speed we do not have to put the entire code that belongs to speed inside the Van class., which makes it possible to reuse the Speed class in multiple applications.

HAS-A Relationship

In Object Oriented feature the users do not need to bother about which object is doing the real work. To achieve this, the Van class hides the implementation details from the users of the Van class. SO basically what happens is the users would ask the Van class to do a certain action and the Vann class will either do the work by itself or ask another class to perform the action.

A very important fact to remember is that Java only supports only single inheritance. This means that a class cannot extend more than one class. Therefore following is illegal:

```
public class extends Animal, Mammal{}
```

However a class can implement one or more interfaces. This has made Java get rid of the impossibility of multiple inheritance

Lab Tasks

```
class Animal {  
}  
  
class Mammal extends Animal {  
}  
  
class Reptile extends Animal {  
}  
  
public class Dog extends Mammal {  
  
    public static void main(String args[]) {  
        Animal a = new Animal();  
        Mammal m = new Mammal();  
        Dog d = new Dog();  
  
        System.out.println(m instanceof Animal);  
        System.out.println(d instanceof Mammal);  
        System.out.println(d instanceof Animal);  
    }  
}
```

Lab Tasks

```
class Calculation {
    int z;

    public void addition(int x, int y) {
        z = x + y;
        System.out.println("The sum of the given numbers:"+z);
    }

    public void Subtraction(int x, int y) {
        z = x - y;
        System.out.println("The difference between the given numbers:"+z);
    }
}

public class My_Calculation extends Calculation {
    public void multiplication(int x, int y) {
        z = x * y;
        System.out.println("The product of the given numbers:"+z);
    }

    public static void main(String args[]) {
        int a = 20, b = 10;
        My_Calculation demo = new My_Calculation();
        demo.addition(a, b);
        demo.Subtraction(a, b);
        demo.multiplication(a, b);
    }
}
```

```
class Teacher {
    String designation = "Teacher";
    String collegeName = "Beginnersbook";
    void does(){
        System.out.println("Teaching");
    }
}

public class PhysicsTeacher extends Teacher{
    String mainSubject = "Physics";
    public static void main(String args[]){
        PhysicsTeacher obj = new PhysicsTeacher();
        System.out.println(obj.collegeName);
        System.out.println(obj.designation);
        System.out.println(obj.mainSubject);
        obj.does();
    }
}
```

Final Lab Task

- Create a class Plants that has child classes of Trees and Shrubs.
- Create a Sub class of Trees as Fruit bearing tress. Identify attributes and methods of each class.
- Instantiate the fruit bearing tree class and show its attributes.
- Show the classes in the form of a tree as well